

SERVER · DEVOPS ENGINEER / 김지오

시스템을 설계하는 데서 끝내지 않고, 더 나은 구조로 계속 발전시킵니다.

문제의 원인을 끝까지 추적하고, 반복되는 과정은 자동화하며,
운영에서 얻은 경험을 기록하고 다음 개선으로 연결하는 엔지니어입니다.

Infrastructure Architecture

IaC

Automation

Monitoring

SELECTED WORK

- 01 DevOps Engineer Intern**
Infrastructure · Automation · Monitoring
- 02 IaC 기반 HA 지향 Kubernetes 클러스터**
Terraform · Ansible · 구축 자동화
- 03 축제 서비스 Cardinal**
개발 · 배포 · 실제 운영

HOW I ENGINEER

기술보다 문제를 먼저 정의하고, 운영을 통해 구조를 검증합니다.

인턴십을 통해 Kubernetes 기반 인프라 구축·자동화·모니터링과 실제 운영 환경을 경험했습니다.

이후 실제 사용자를 대상으로 운영되는 서비스에 직접 설계한 아키텍처를 적용하며,

운영에서 발견한 문제를 다시 구조 개선으로 연결해왔습니다.

OPERATING PRINCIPLES

DEFINE

기술보다 문제를 먼저 정의합니다.

무엇을 쓸지보다 왜 필요한지를
먼저 고민합니다.

DESIGN

문제를 구조로 해결합니다.

이전 경험과 제약 조건을 바탕으로
아키텍처를 능동적으로 설계합니다.

VALIDATE

실제 운영으로 검증합니다.

동작한다는 것에 만족하지 않고 실제
사용자와 운영 환경에서 확인합니다.

IMPROVE

경험을 다음 설계로 연결합니다.

문제와 해결 과정을 기록하고 다음 프로젝트의
개선점으로 남깁니다.

HOW I LEAD

먼저 이해하고, 먼저 움직이며, 함께 움직일 수 있는 구조를 만듭니다.

내가 맡은 일을 깊이 이해하는 것에서 책임감 있는 리더십이 시작된다고 생각합니다.

질문하고 의견을 제시하며, 소통과 역할의 문제를 발견하면 팀이 더 잘 움직일 수 있는 방식으로 계속 개선해왔습니다.

COMMUNICATION

멋쟁이사자처럼 서강대 회장, 리더의 책임감

소통이 원활하지 않았던 초기 운영을 돌아보며 전달과 피드백 방식을 개선했습니다.
이를 통해 역할과 진행 상황이 자연스럽게 공유되는 협업 구조를 만들어갔습니다.

INITIATIVE

익숙한 방식보다, 목적에 맞는 방식을 선택했습니다.

기존의 동아리 프로그램이 창업 동아리의 목적과 맞지 않는다고 판단해,
구성원들이 실제 창업 과정을 경험할 수 있도록 프로그램의 방향과 운영
프로세스를 재설계했습니다.

시스템을 구성하는 일부터, 코드로 움직이게 만드는 일까지 해왔습니다.

● Core ● Used

Backend

REST API 구현 · 조회 쿼리 최적화

- Spring Boot
- REST API
- SQL
- QueryDSL
- Caffeine

Cloud / Infrastructure

VPC 네트워크 구성 · ALB/ASG 기반 확장 구조 설계

- AWS
- EC2
- VPC
- ALB
- NLB
- ASG
- Route 53
- IAM
- NAT
- Custom AMI

Container / Orchestration

Staging Docker 구성 · Production Kubernetes 운영

- Docker
- Kubernetes

CI/CD

브랜치별 빌드·배포 자동화 · Blue-Green 기반 무중단 배포

- GitHub Actions
- Argo CD
- Blue-Green Deployment
- Jenkins
- Watchtower

Monitoring / Observability

사내 모니터링 서버 구축 · 운영 대시보드 설계

- Prometheus
- Grafana

IaC / Automation

Kubernetes 클러스터 부트스트랩 자동화 · 인프라 구성 코드화

- Ansible
- Terraform

Linux / Server

OS 설치 및 서버 초기 구성 · BMT 폐쇄망 환경 재현

- Ubuntu
- Rocky Linux
- Amazon Linux
- RHEL

1. AI Infra & DevOps Engineer Intern

인프라를 직접 다루며, 더 안정적인 운영 방식을 만들어왔습니다.

Linux · Kubernetes · Ansible · Prometheus/Grafana 기반 AI 인프라 구축 · 자동화 · 모니터링 실무



사내 인프라의 구축과 운영을 직접 담당하며 반복되는 작업과 관리상의 비효율을 자동화와 표준화로 개선했습니다.
고객사 BMT에서는 실제 현장 환경에 맞춘 인프라 구축과 기술 대응을 수행했습니다.

01 / STANDARDIZE

Proxmox VM 환경 표준화

6종 OS 템플릿 구축 · 반복 설치 제거 · VM 구축 시간 단축

02 / OBSERVE

수십 대 서버 통합 모니터링 인프라 구축

78대 서버 중앙 모니터링 · Prometheus/Grafana · GPU 지표 수집

03 / AUTOMATE

서버 사양 수집 자동화 워크플로우 구축

78대 서버 정보 자동 수집 · Ansible · 약 5시간의 반복 업무 제거

04 / COORDINATION

국내 대형 손해보험사 BMT 인프라 구축

폐쇄망 환경 재현 · Kubernetes 배포 환경 구성 · GPU 인프라 현장 대응

PREVIEW

사내 Proxmox VM 환경 표준화

같은 VM 환경을
매번 처음부터 구축하고 있었습니다.

사내 Proxmox 환경에서 개발·검증용 VM을 생성할 때마다 OS
설치부터 초기 설정까지 같은 과정을 반복해야 했습니다.
특히 GPU를 사용하는 환경에서는 OS와 Kernel, NVIDIA
Driver 조합까지 다시 확인해야 해 VM 한 대를 준비하는 데 약
15분이 필요했습니다.

VM 생성	vCPU · Memory · Disk 등 기본 리소스 설정
OS 환경	Ubuntu · Rocky Linux 등 목적에 맞는 OS 설치
GPU 환경	NVIDIA Driver · Kernel 호환성 확인

ROLE Proxmox VM 환경 분석 · 표준화 방식 설계 · Template 구축 및 검증

DIRECTION

매번 환경을 새로 만드는 대신, 검증된 실행 환경 자체를
재사용하는 방향으로 표준화했습니다.

VM 구축 시간

약 15분 → 약 5분

약 67% 단축

표준 OS 템플릿

0종 → 6종

Ubuntu 2종 · Rocky Linux 4종

환경 불일치 재작업·문의

발생 → 0건

동일한 검증 환경 재현

PROBLEM

VM은 만들 수 있었지만, 같은 환경을 반복해서 준비하는 과정에 운영 관점의 비효율이 있었습니다.

1

VM마다 OS 설치를 반복해야 한다

새로운 VM이 필요할 때마다 ISO를 연결하고 OS를 설치한 뒤 기본 설정을 다시 수행해야 했습니다.

RISK VM 수가 늘어날수록 동일한 구축 작업과 대기 시간이 반복됨

2

실행 환경이 사람과 작업마다 달라질 수 있다

동일한 용도의 VM이라도 OS 버전과 초기 설정이 달라질 수 있어 환경 차이에서 발생하는 문제를 다시 확인해야 했습니다.

RISK 환경 불일치로 인한 재작업과 트러블슈팅 비용 증가

3

GPU 환경은 호환성까지 다시 확인해야 한다

NVIDIA Driver는 Kernel과의 호환성을 함께 고려해야 해, 검증된 조합을 매번 다시 구성하는 방식은 운영 부담이 컸습니다.

RISK Driver · Kernel 조합 차이로 실행 환경의 안정성이 흔들릴 수 있음

STACK Proxmox VE · Ubuntu · Rocky Linux · NVIDIA Driver

SOLUTION

검증된 VM 환경의 재사용 구조 설계

단순히 Template을 만든 것이 아니라,
어떻게 재사용할지를 기준으로 구조를 정했습니다.

01	02	03
검증 환경의 재사용	독립 실행 환경	GPU 조합의 고정

01	DECISION cloud-init 없이 검증된 VM 환경을 Template으로 전환	WHY Proxmox 도입 초기 단계였기 때문에 자동 초기화 체계를 먼저 도입하기보다, 이미 검증된 OS·Driver 환경을 빠르게 재사용하는 것을 우선했습니다. Clone 이후 IP와 리소스는 수동으로 조정했습니다.	TRADE-OFF 빠르게 표준 환경을 만들 수 있었지만, IP·리소스 관리에 사람의 개입이 남았습니다.
02	DECISION Linked Clone 대신 Full Clone 선택	WHY Linked Clone은 원본 Template의 디스크에 의존하기 때문에, 각 VM을 독립적인 개발·검증 환경으로 사용할 수 있도록 Full Clone 방식을 선택했습니다. Template의 기본 디스크 용량은 최소한으로 구성했습니다.	TRADE-OFF Linked Clone보다 Storage 사용량은 늘지만, 원본 Template과 독립적인 실행 환경을 확보했습니다.
03	DECISION NVIDIA Driver 설치 Kernel 버전을 검증된 조합으로 고정	WHY NVIDIA Driver는 Kernel Module과 함께 동작하므로 Kernel이 임의로 업데이트되면 기존 Driver와 호환되지 않을 수 있습니다. 이를 막기 위해 Kernel 패키지를 고정하였습니다.	TRADE-OFF 최신 Kernel을 즉시 적용하는 대신, 검증된 GPU 실행 환경의 안정성과 재현성을 우선했습니다.

PROCESS & RESULT

Troubleshooting & Insights

설치 실패를 단순 오류로 보지 않고, Kernel과 Package의 관계까지 추적했습니다.

상황 Template을 구성하기 위해 NVIDIA Driver를 설치하던 중, 실행 중인 Kernel에 맞는 Module을 빌드하지 못하며 설치가 실패했습니다.

원인 추적		해결 방법	
01	실행 Kernel 확인 현재 Kernel과 Module 빌드 조건을 확인했습니다.	03	동일 버전 Package 탐색 필요한 Package를 찾아 Build 환경을 맞췄습니다.
02	Build Package 비교 kernel-devel · headers 버전 일치 여부를 확인했습니다.	04	Driver 재설치 일치한 환경에서 정상 동작을 확인했습니다.

RESULT

Rocky Linux 8.6의 지원 종료로 실행 Kernel과 일치하는 Build Package를 기본 Repo에서 확보할 수 없었습니다.

기본 Repository에 없는 Kernel Build Package를 직접 탐색·설치해 NVIDIA Driver의 Kernel Module 빌드를 정상화했습니다.

01

오류 메시지보다 의존 관계를 따라가게 되었습니다.

하나의 설치 실패도 해당 Package만 보는 것이 아니라 Kernel · Module · Repository까지 연결된 실행 환경 전체를 확인해야 한다는 점을 배웠습니다.

02

표준화는 자동화로 가는 다음 문제를 만듭니다.

Template로 동일 환경을 재현한 뒤에도 IP와 CPU·Memory·Disk 설정은 남았습니다. 다음 단계에서 cloud-init과 IaC가 필요함을 확인했습니다.

PREVIEW

수십 대 서버를 한 화면에서 파악하는 통합 모니터링 인프라

서버의 이상 징후를 발견할 수 있도록, 관측 가능한 운영 환경을 만들었습니다.

사내와 데이터센터의 다수 서버를 관리하고 있었지만, 각 서버의 자원 상태를 한 화면에서 확인할 수단이 없었습니다. 비정상적인 자원 점유와 이상 징후를 제때 발견하기 어려운 상황에서, 통합 모니터링 환경의 구축을 맡았습니다.

운영 범위

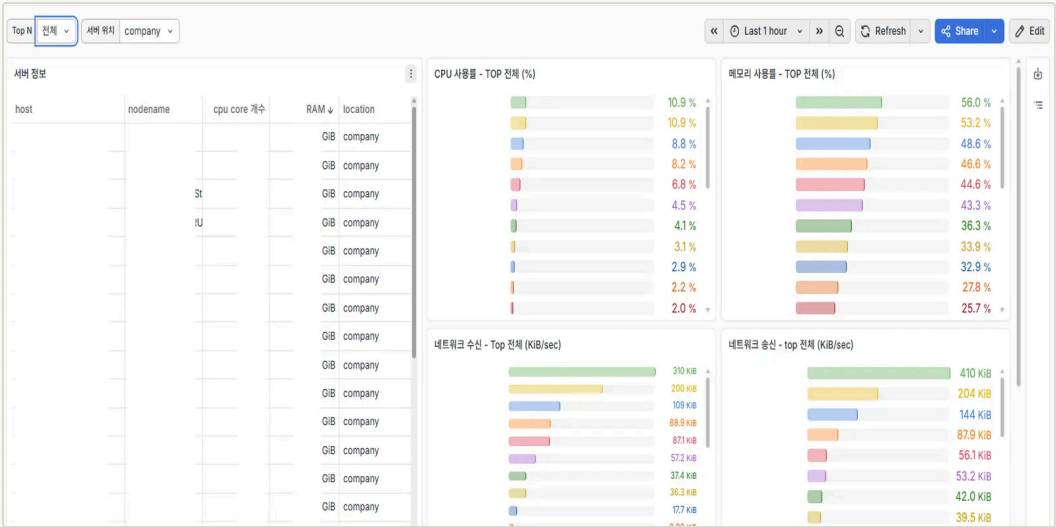
대상 서버
사내 · 데이터센터
서버 70여 대

관측 지표

CPU · Mem · Network · GPU

운영 목표

이상 서버 식별 · 신규 서버 편입
재사용 가능한 배포



GRAFANA · SERVER OBSERVABILITY

ROLE 모니터링 아키텍처 설계 · Ansible 기반 수집 환경 배포 자동화
Grafana 대시보드 템플릿 제작

STACK Ansible · Prometheus · node_exporter · Grafana · PostgreSQL · Linux

PROBLEM

서버는 많았지만, 상태를 비교하고 이상을 빠르게 찾는 공통의 관측 지점이 없었습니다.

01 서버 자원 상태가 각 서버에 흩어져 있었다

CPU · 메모리 · 네트워크 상태를 서버별로 접속해 확인해야 했습니다.

RISK 이상 징후 발견 지연 · 개별 확인에 의존한 운영 판단

02 수집 환경을 수십 대에 반복 적용해야 했다

node_exporter 설치와 기본 설정을 서버마다 따로 수행해야 했습니다.

RISK 수동 설치 · 설정 편차 · 신규 서버 편입 지연

03 GPU 지표는 별도 수집 경로가 필요했다

일반 시스템 지표와 달리 GPU 상태는 별도 방식으로 수집해야 했습니다.

RISK 통합 대시보드에서 GPU 상태를 함께 판단하기 어려움

ARCHITECTURE

Ansible 기반 IaC로 표준화한 서버 관측 구조

Ansible 기반 IaC로 수집 환경을 표준화하고, PostgreSQL 적재를 통해 GPU까지 관측 범위를 확장했습니다.

DATA FLOW

01 Inventory 등록 서버 목록 · 접속 정보	02 Playbook 실행 설치 · 설정 · OS 분기	03 일반 시스템 지표 CPU · Mem · Network → Prometheus	04 Grafana 통합 확인 두 수집 경로 비교
		03 GPU · Process 지표 Ansible 스냅샷 → PostgreSQL	

일반 시스템 지표와 GPU · Process 지표는 서로 다른 수집 경로를 거쳐 하나의 Grafana 대시보드에서 확인했습니다.

01

DEPLOYMENT

Agentless 배포를 선택

SSH와 Ansible Playbook으로 설치와 설정을 일괄 적용

역등성 있는 실행으로 78대와 이후 추가 서버에도 같은 상태를 재사용했습니다.

02

COMPATIBILITY

OS 계열별 방화벽 처리 분기

Debian · RedHat 계열의 도구 차이를 Ansible에서 구분

서버별 환경 차이를 숨기지 않고, 수집 포트가 막히지 않도록 계열별로 처리했습니다.

03

COLLECTION

연속된 스냅샷 적재를 통한 GPU 수집

Ansible로 GPU · Process 정보를 수집 후 PostgreSQL에 적재

Prometheus 기반 실시간 수집과 다른 방식으로 GPU 상태를 저장 · 조회하는 구조를 검증했습니다.

운영 결과

서버 자원 가시성

78대 통합 대시보드

Grafana 한 화면에서 실시간 확인

신규 서버 모니터링 편입

약 20분 → 약 2분

Inventory 등록 후 Playbook 실행

TROUBLESHOOTING & INSIGHTS

데이터 수집으로 인해 모니터링 서버의 저장 여력은 한 달 안에 한계에 도달했습니다.

5분마다 GPU 지표를 수집하자 하루 약 4GB가 쌓였습니다.
정보 최신성과 저장 용량 사이의 균형을 고민해야 했습니다.

01

STORAGE

고빈도 스냅샷이 하루 약 4GB 쌓였다

관 찰 전체 GPU 지표를 수집하면서 5분마다 1,000건 이상의
지표 값이 디스크에 기록됐습니다.

판 단 GPU 이상은 CPU·메모리 등 시스템 지표와 함께 판단할 수
있다고 보고, GPU 수집 주기를 15분으로 조정했습니다.

TRADE-OFF GPU 정보의 즉시성은 낮아졌지만, 모니터링 서버의
저장 여력을 확보했습니다.

02

CONSISTENCY

제거된 GPU 정보가 Grafana에 남았다

관 찰 GPU가 제거·교체된 뒤에도 이전 정보가 남아 실제 구성과
화면이 달라졌습니다.

판 단 gpu 테이블에 활성 상태 컬럼을 추가하고, Ansible 수집 시
제거된 GPU는 false로 갱신했습니다.

RESULT 현재 구성에 존재하는 GPU만 표시해 실제 서버 구성과
대시보드의 정합성을 유지했습니다.

RESULT

78대 서버의 상태와 GPU·프로세스별 점유율을 대시보드에서 한눈에 확인할 수 있게 됐습니다.

Inventory에 서버만 추가하면 동일한 배포 · 수집 · 대시보드 구조를 적용할 수 있도록 구성해, 사내 운영 자산으로 남겼습니다.

INSIGHTS

운영 자동화는 일회성 실행이 아니라, 확장을 전제로 한 구조여야 합니다.

사내 운영 자산으로 남기려면, 서버를 쉽게 추가·제거·변경할 수 있는 구조와 이를 뒷받침하는 상세한 운영 문서가 함께 필요하다는 점을 깨닫고, 이를 구조와 문서에 반영했습니다.

PREVIEW

서버 구성 관리 자동화 시스템 구축

수작업으로 진행되던 서버 구성 관리를 자동화 및 정형화를 통해 정보 불일치를 해결하였습니다.

약 1년간 방치된 서버 1대와 정보 불일치 4건을 발견하고 보고하였습니다.

	A	B	C	D	E	F	G	
1	Server	Hostname	Internal IP	External IP	Gateway	OS	CPU Model	Phy
2	sogang038	master-1	163.239.77.98	163.239.77.98	163.239.77.1	Ubuntu 25.04	Intel(R) Xeon(R) CPU E5-2660 v3 @ 2.60GHz	8
3	sogang039	master-2	163.239.77.99	163.239.77.99	163.239.77.1	Ubuntu 25.04	Intel(R) Xeon(R) CPU E5-2660 v3 @ 2.60GHz	8
4	sogang040	master-3	163.239.77.100	163.239.77.100	163.239.77.1	Ubuntu 25.04	Intel(R) Xeon(R) CPU E5-2660 v3 @ 2.60GHz	8
5	sogang041	worker-1	163.239.77.101	163.239.77.101	163.239.77.1	Ubuntu 25.04	Intel(R) Xeon(R) CPU E5-2660 v3 @ 2.60GHz	8
6	sogang042	worker-2	163.239.77.102	163.239.77.102	163.239.77.1	Ubuntu 25.04	Intel(R) Xeon(R) CPU E5-2660 v3 @ 2.60GHz	8
	sogang043	worker-3	163.239.77.103	163.239.77.103	163.239.77.1	Ubuntu 25.04	Intel(R) Xeon(R) CPU E5-2660 v3 @ 2.60GHz	8

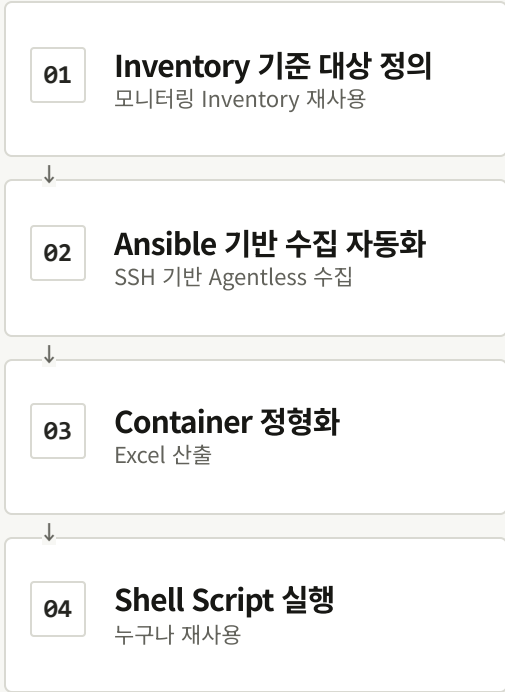
사내 정보 보호를 위해 개인 서버 예시로 대체하였습니다.

상황 및 운영 결과

70여 대 사양 수집 시간	약 4시간 수작업 → 약 10분 자동 스크립트 실행	70여 대의 주기 확인 작업을 단일 스크립트 실행으로 대체
관리 대장 불일치 발견	미안자 → 4건 발견	OS 정보·비밀번호·SSH 포트 정보 불일치 4건을 확인
방치된 GPU 서버 발견	미안자 → 1건 발견	RTX 4090 2대가 장착된 GPU 서버를 확인
관리 문서 갱신	약 3개월 미갱신 → 필요 시 즉시 갱신	단일 실행으로 최신 서버 구성 정보를 다시 수집해 관리 문서를 갱신

PROCESS

담당자가 쉽게 사용할 수 있도록 실행 과정을 기획하고 구현하였습니다.



WHY 01	설계 비용을 최소화했습니다.	모니터링 프로젝트를 진행하며 정리한 Inventory 구조를 적용해, 대상 정의와 초기 설계에 드는 비용을 줄였습니다.
WHY 02	대상 서버마다 패키지를 설치하는 것은 운영 부담이었습니다.	별도 Agent 설치 없이 SSH로 접속해 OS · IP · Hostname · CPU · Mem · Disk 정보를 일괄 수집했습니다.
WHY 03	Excel 생성 환경은 컨테이너로 분리했습니다.	여러 패키지가 필요한 정형화 작업을 컨테이너에 격리해, 운영 서버에 설치 부담과 의존성을 남기지 않았습니다.
WHY 04	Ansible을 몰라도 실행할 수 있게 했습니다.	Shell Script 한 번으로 결과를 생성하도록 구성해, 다른 구성원도 쉽게 활용할 수 있는 사내 운영 자산으로 남겼습니다.

PREVIEW

폐쇄망 환경에서 vLLM 기반 보험금 지급 금액 심사 인프라 설계

국내 대형 손해보험사 BMT에서
총 7명 규모의 팀 인프라를 단독 전담했습니다.

제약 조건

고객사 조건 RHEL · Kubernetes 1.34 · Cilium · containerd · 폐쇄망 환경 · PuTTY 접속

개발팀 조건 안정적인 백엔드 애플리케이션과 vLLM 환경 구축

ROLE BMT 인프라 단독 전담 · Kubernetes 리소스 · 네트워크 구조 설계
AWS 사전 검증 환경 구축 및 현장 배포 대응

STACK Kubernetes · vLLM · Gemma4-31B-it · AWS H100 · Linux (RHEL)

설계 포인트

01

온프레미스 H100 검증 환경이 없었다

RTX 5000 4장 구성으로 검증하면 현장 H100 환경과 결과가
달라질 수 있다고 판단했습니다.



AWS H100 사전 검증 환경 구성

AWS H100 인스턴스를 예약해 고객사 환경과 유사한 사전
검증 환경을 구성했습니다.

02

latest 태그 이미지의 반입 기준이 불확실했다

개발팀이 사용하던 Gemma 이미지의 태그가 latest로
지정되어 있었습니다.



SHA 일치 이미지 버전 고정

USB 반입 전 Docker Hub에서 SHA가 일치하는 정확한
이미지 버전을 찾아 고정했습니다.

03

개발팀이 테스트를 쉽게 할 수 있도록 해야 했다

이미지가 교체되더라도 쉽게 테스트할 수 있는 환경을 조성해야
했습니다.



K8s Deployment · Service 구성

Deployment와 Service 리소스를 구성해 이미지 교체와
서비스 접근 과정을 단순화했습니다.

PROCESS

폐쇄망에 반입하기 전에, 현장 조건을 외부에 그대로 만들었습니다.

01 검증 환경 구축 Rocky 9.4 · 온프레미스 검증 · SFTP	WHY 01 비용이 큰 H100 대신 온프레미스에서 먼저 검증했습니다. 현장 RHEL은 유료 라이선스라 가장 가까운 Rocky 9.4로 테스트 서버를 새로 설치해 검증하였습니다.
02 이미지 버전 고정 inspect · digest · 특정 태그	WHY 02 현장과 같은 이미지를 반입할 수 있게 했습니다. 사용 중인 vLLM 컨테이너를 inspect해 image digest를 확인하고, 동일 digest에 대응하는 특정 태그로 고정했습니다.
03 Kubernetes 설계 vLLM · Backend 분리 · ClusterIP	WHY 03 파드 IP 변화와 무관한 내부 통신을 만들었습니다. vLLM과 Backend를 별도 파드로 분리하고 ClusterIP Service와 고정 DNS 이름으로 연결했습니다.
04 H100 용량 확보 시드니 리전 · 용량 예약	WHY 04 2일 안에 쓸 수 있는 GPU를 확보했습니다. 확보 가능한 H100이 없어 AWS·Azure·SCP·NCP를 확인한 뒤, 지연시간이 짧은 AWS 시드니 리전에 용량을 예약하고 아웃바운드를 열어 RHEL에 고객사 조건의 Kubernetes를 설치했습니다.
05 당일 변경 대비 소스코드 마운트 · mount / nomount	WHY 05 당일 코드 변경까지 대비했습니다. 개발팀의 당일 소스 코드 변경 가능성이 생겨, 현장 재빌드 없이 대응하도록 소스코드를 마운트하는 매니페스트를 따로 작성했습니다.
06 폐쇄망 재현 · 최종 검증 아웃바운드 차단 · SSH만 허용 · SFTP	WHY 06 현장과 같은 폐쇄망에서 최종 검증까지 마쳤습니다. H100 서버의 아웃바운드를 차단해 SSH 포트를 제외한 폐쇄망 환경을 재현하고, SFTP로 완성된 반입물을 AWS H100 서버에 반입한 뒤 최종 검증을 진행하였습니다.

TROUBLESHOOTING & RESULT

현장 변수는 사전 검증만으로 끝나지 않았고, 실제 구성에 맞춰 다시 판단해야 했습니다.

01

FILE INTEGRITY

USB 반입 파일의 실제 용량이 달랐습니다.

관찰 현장에서 확인한 모델 파일은 예상한 59GB가 아니라 1.7GB로 표시됐습니다.

원인 전날 AWS H100 서버에서 최종 검수를 마친 뒤 최신 파일만 USB에 담으려 SFTP로 복사했는데, 전송이 끝나기 전에 완료 여부를 확인하지 않고 분리했습니다.

대응 불변이 확인된 safetensor를 제외한 파일만 다시 동기화하고, 서버에서 정상 파일을 재확인한 뒤 USB를 다시 반입했습니다.

결과 모델 파일을 정상 상태로 복구해 현장 배포 기준을 맞췄습니다.

02

GPU RESOURCE

vLLM 파드의 GPU를 Index로 지정할 수 없었습니다.

관찰 현장 H100 GPU가 MIG로 분할돼 있어, 기존 GPU Index 기반 리소스 요청은 사용할 수 없었습니다.

원인 고객사가 GPU 할당 방식을 따로 언급하지 않고 H100 한 장을 제공한다고만 해, Index 방식일 것으로 가정했습니다.

대응 GPU Index와 CUDA_VISIBLE_DEVICES 방식 대신, 배정된 MIG 프로파일 리소스를 requests·limits로 요청하도록 Manifest를 변경했습니다.

결과 vLLM(Gemma4-31B-it) 파드의 정상 기동을 로그로 확인했습니다.

RESULT

지정된 폐쇄망 조건에서 vLLM 기반 보험금 지급 금액 심사 시스템의 배포와 동작을 검증했습니다.

사전 환경 재현·이미지 버전 고정·반입 파일 검증·MIG 리소스 설계를 통해 현장 변수에도 대응 가능한 BMT 인프라를 구성했습니다.

INSIGHT

최악의 상황을 가정하고 준비해야 합니다.

현장에는 예상치 못한 변수가 생기므로, 반입물의 전송 완료와 무결성을 직접 확인하고 달라질 수 있는 구성에는 대안을 함께 준비해야 합니다.

아는 것을 팀에 먼저 공유해야 합니다.

현장에서 BMT 실행을 매번 curl로 호출하며, API 형태의 실행 방식을 미리 제안했다면 팀 전체가 더 수월했겠다고 느꼈습니다.

귀찮아서 시작했고, 만들고 나서 **보안까지** 좁혔습니다.

2. IaC 기반 HA 지향 Kubernetes 클러스터

AWS 인프라 프로비저닝 · Kubernetes 클러스터 구축 자동화

업무에서 반복하던 쿠버네티스 프로비저닝을, **개인 프로젝트로 직접 자동화했습니다.**
설치를 **Ansible**로 자동화했고, 이를 검증할 환경이 필요해 AWS 인프라를 **Terraform**으로 코드화했습니다.
만들고 나서야 드러난 보안 문제를 **v1에서 v3까지** 단계적으로 좁혀갔습니다.

STACK Terraform · Ansible · Kubernetes · AWS (EC2 · VPC · NLB · SSM) · Claude

RESULT **약 1시간 → 약 5분**
HA 클러스터 구축 시간 · 약 92% 단축

39개의 리소스
control-plane 3 · worker 2 · terraform apply 1회로 생성

다운타임 0%
control-plane 1대 중지 시

SECURITY HARDENING

한 번에 설계하지 못했고, 만들고 나서 보인 문제를 세 단계에 걸쳐 좁혔습니다.

1단계 v1

단일 control-plane에서 Ansible로 Kubernetes 환경을 설치했습니다

- control-plane 1대 · worker 1대
- Kubernetes 버전 · CRI · CNI를 지정해 설치
- 노드 간 통신을 퍼블릭 IP로 연결

설계 포인트

control-plane은 비용이 낮은 Lightsail로, worker는 파드가 올라가는 만큼 수직 확장이 자유로운 EC2로 나눴습니다.

생긴 문제

노드끼리 통신할 때 퍼블릭 IP를 주고받으면서 네트워크 비용이 발생했습니다. 또한 Lightsail은 자체 default VPC에서 돌아가 EC2와 내부 통신을 할 수 없었습니다.

2단계 v2

하나의 VPC로 통합하여 사설 네트워크 통신으로 만들었습니다

- 전 노드를 EC2로 통일하고 커스텀 VPC 구성
- control-plane은 퍼블릭 · worker는 프라이빗 서브넷
- 사설 IP 통신으로 인터넷 경유 제거

설계 포인트

네트워크가 갈려 생긴 요금과 통신 문제를 없애려면 한 VPC로 모아야 했고, control-plane은 SSH 접근이 필요해 퍼블릭에 남겼습니다.

남은 문제

control-plane이 여전히 퍼블릭에 노출돼 SSH 인바운드가 열려 있었고, 이 control-plane을 거치면 프라이빗 서브넷의 worker까지 닿을 수 있었습니다.

3단계 v3

클러스터를 Private Subnet에 배치하고 control-plane을 다중화했습니다

- 전 노드 프라이빗 서브넷
- 관리 접근은 SSM
- 인바운드는 Reverse Proxy
- 아웃바운드는 NAT Instance
- 출처 차단은 NACL
- control-plane 3대와 내부 NLB로 단일 엔드포인트

결과

control-plane 3대 중 1대가 멈춰도 클러스터가 유지되는 구성을 완성했습니다. 다만 단일 AZ라 가용 영역 장애까지는 견디지 못합니다.

AUTOMATION & RESOURCES

설치는 변수로 받고,
막힐 때마다 리소스를 하나씩 더했습니다.

- Terraform

콘솔에서 하나씩 만들면 빠뜨리기 쉬워,
AWS 리소스를 코드로 모아 한 번에 다루도록 했습니다.
- Ansible

프로젝트의 목표 자체가 설치 자동화였기에,
모든 노드에 같은 절차가 적용되도록 만들었습니다.

Ansible	역할을 시나리오마다 새로 쓰지 않고 변수로 조합하도록 만들어, 플레이북 수정 없이 원하는 구성을 골라 설치할 수 있게 했습니다.					
	Kubernetes	1.24 이상 · 기본 1.30 pkgs.k8s.io에서 제공하는 범위	Container Runtime	containerd · CRI-O · cri-dockerd	CNI	Calico · Flannel · Cilium
	control-plane	1대 또는 3대 이상	OS	Ubuntu · Rocky 모두 동작	폐쇄망 워커 (v2)	control-plane에서 worker로 scp 아웃바운드가 있는 control-plane이 받아 전달

Terraform

01

SSH 인바운드를 없애 노드에 접근할 수 없다

→ SSM Session Manager

VPC 인터페이스 엔드포인트로 AWS 내부망만 경유

02

프라이빗 서브넷의 애플리케이션에 외부가 닿을 수 없다

→ Reverse Proxy

유일한 인바운드 진입점 · NodePort는 이 SG 출처로만 개방

03

아웃바운드가 없어 쿠버네티스 패키지를 설치할 수 없다

→ NAT Instance

NAT Gateway 대신 저렴한 인스턴스로 비용 절감

04

control-plane 1대가 멈추면 클러스터가 함께 멈춘다

→ 정족수를 고려한 control-plane 3대

1대가 빠져도 etcd 정족수가 유지되는 최소 구성

05

control-plane 3대의 apiserver 주소를 하나로 묶어야 한다

→ 내부 NLB

kubeadm join이 바라볼 고정 엔드포인트 · 사설 IP만 할당

06

특정 출처를 차단해야 하는데 SG는 allow만 가능하다

→ NACL deny 규칙

퍼블릭 서브넷 경계에서 악성 IP를 먼저 거름

TROUBLESHOOTING & INSIGHT

안전하게 만들수록, 제가 접근하고 개발하기도 까다로워졌습니다.

달아둔 경로와 새로 넣은 구성요소가 그대로 작업의 제약이 됐습니다.

01

NLB HAIRPIN

다중 control-plane에서 Kubernetes 설치 이후, kubectl 명령이 간헐적으로 실패했습니다.

모든 kube-system 파드가 정상이었고 Linux 명령도 잘 실행돼 인스턴스 문제는 아니라고 판단했습니다. v3에서 새로 추가한 NLB·NAT Instance·Reverse Proxy 중 Kubernetes와 직접 닿는 것은 NLB뿐이라 보고 관련 이슈를 찾아보았습니다.

kubeconfig가 NLB 엔드포인트를 향하고 있어, kubectl 명령을 날리던 control-plane이 NLB의 타겟이면서 동시에 클라이언트가 되는 헤어핀 문제라는 것을 알게 되었습니다. 클라이언트 IP 보존이 켜져 있으면 자기 자신으로 되돌아온 요청에 응답하지 못하기 때문에, 타겟 그룹에서 이 옵션을 끄고 다시 검증했습니다.

02

MEMORY

Kubernetes 설치 파일을 옮기던 중 control-plane 서버가 다운됐습니다.

worker가 인터넷에 나갈 수 없어 control-plane이 받아 전달하는 구조였습니다.

Ansible fetch가 파일을 메모리에 통째로 올리면서, kube-system 파드가 이미 떠 있던 control-plane이 이를 견디지 못했습니다.

따라서 메모리에 쌓지 않고 control-plane에서 worker로 곧장 보내는 scp로 변경하였습니다.

03

SSH ACCESS

Ansible이 프라이빗 노드에 접속하지 못했습니다.

Ansible은 SSH 기반이라 퍼블릭 IP도 SSH 인바운드도 없는 노드에는 붙을 수 없었습니다.

인벤토리에 ProxyCommand를 걸어 SSH 연결을 SSM 세션 위로 태웠고, 접속 대상도 IP가 아닌 인스턴스 ID로 지정해 인바운드를 열지 않고 관리하도록 했습니다.

INSIGHT

만든 것으로 끝내지 않아야 합니다.

실제로 동작하는지 증명하기 위해, Cardinal 프로젝트를 이 위에 올려보기로 했습니다.

확장성과 보안을 함께 설계해야 합니다.

단일 AZ처럼 남겨둔 한계를 다음에는 넘어서기 위해 AWS SAA를 준비하였습니다.